

Chapter 21 Generics



Objectives

- ❑ To know the **benefits** of **generics** (§21.2).
- ❑ To use/ declare **generic classes** and **interfaces** (§21.3).
- ❑ To understand why generic types can improve reliability and readability (§21.3).
- ❑ To declare and use **generic methods** (§21.4).



Why Do You Get a Warning?

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList list =  
            new java.util.ArrayList();  
        list.add("Java Programming");  
    }  
}
```



A compile warning on this line occurs



Fix the Warning

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList<String> list =  
            new java.util.ArrayList<String>();  
        list.add("Java Programming");  
    }  
}
```



No compile warning on this line.



What is Generics?

- ✓ *Generics* is the capability to parameterize types.
- ✓ The idea is to allow type (Integer, String, ... etc., and user-defined types – **not primitives like *int*, *char* or *double***) to be a parameter to **methods, classes, and interfaces**.
- ✓ You can define a **class**, an **interface** or a **method** with **generic types** that can be substituted using **concrete types** by the compiler.

For example:

- ArrayList
- Java.lang.Comparable
- ✓ To create an instance of a generic class:

```
BaseType <Type> obj = new BaseType <Type>();
```



Why Generics?

- ✓ The key benefit of generics is to enable errors to be detected at compile time rather than at runtime.
- ✓ A generic class/interface or method permits you to specify allowable types of objects that the class/interface or method may work with.
- ✓ If you attempt to use the class or method with an incompatible object, a compile error occurs.



Generic ArrayList in JDK 1.5

java.util.ArrayList

```
+ArrayList()  
+add(o: Object): void  
+add(index: int, o: Object): void  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): Object  
+indexOf(o: Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+size(): int  
+remove(index: int): boolean  
+set(index: int, o: Object): Object
```

(a) ArrayList before JDK 1.5

java.util.ArrayList<E>

```
+ArrayList()  
+add(o: E): void  
+add(index: int, o: E): void  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int): E  
+indexOf(o: Object): int  
+isEmpty(): boolean  
+lastIndexOf(o: Object): int  
+remove(o: Object): boolean  
+size(): int  
+remove(index: int): boolean  
+set(index: int, o: E): E
```

(b) ArrayList since JDK 1.5



No Casting Needed

```
ArrayList<Double> list = new ArrayList<>();  
list.add(5.5);  
// 5.5 is automatically converted to new Double(5.5)  
list.add(3.0);  
// 3.0 is automatically converted to new Double(3.0)  
Double doubleObject = list.get(0);  
// No casting is needed  
double d = list.get(1);  
// Automatically converted to double
```



Generic Class Example

```
public class Test<T> {  
    T obj; //An object of type T is declared  
    Test(T obj) { this.obj = obj; } //Constructor  
    public T getObject() { return this.obj; }  
}  
  
public class Main {  
    public static void main(String[] args){  
        Test<Integer> iObj = new Test<Integer>(15); //Instance of Integer type  
        System.out.println(iObj.getObject());  
        Test<String> sObj = new Test<String>("GeeksForGeeks"); //Instance of String type  
        System.out.println(sObj.getObject());  
    }  
}
```



Generic Class Example

```
public class Test<T> {  
    T obj; //An object of type T is declared  
    Test(T obj) { this.obj = obj; } //Constructor  
    public T getObject() { return this.obj; }  
}  
  
public class Main {  
    public static void main(String[] args){  
        Test<Integer> iObj = new Test<Integer>(15); //Instance of Integer type  
        System.out.println(iObj.getObject());  
        Test<String> sObj = new Test<String>("GeeksForGeeks"); //Instance of String type  
        System.out.println(sObj.getObject());  
    }  
}
```

15

GeeksForGeeks

Generic Method Example

```
public class Test {  
    static <T> void genericDisplay(T element){  
        // A Generic method example  
        System.out.println(element.getClass().getName() + " = " + element);}  
    public static void main(String[] args){  
        // Calling generic method with Integer argument  
        genericDisplay(11);  
        // Calling generic method with String argument  
        genericDisplay("GeeksForGeeks");  
        // Calling generic method with double argument  
        genericDisplay(1.0);  
    }  
}
```



Generic Method Example

```
public class Test {  
    static <T> void genericDisplay(T element){  
        // A Generic method example  
        System.out.println(element.getClass().getName() + " = " + element);}  
    public static void main(String[] args){  
        // Calling generic method with Integer argument  
        genericDisplay(11);  
        // Calling generic method with String argument  
        genericDisplay("GeeksForGeeks");  
        // Calling generic method with double argument  
        genericDisplay(1.0);  
    }  
}
```

java.lang.Integer = 11
java.lang.String = GeeksForGeeks
java.lang.Double = 1.0



Generic Interfaces

```
package java.lang;
```

```
public interface Comparable<T> {  
    public int compareTo(T o);  
}
```

- T or E represent a formal generic type
- you can replace it with an actual concrete type
- The type parameters naming conventions are important to learn generics thoroughly. The common type parameters are as follows:
 - T Type
 - E Element
 - K Key
 - N Number
 - V Value



Prior Situation

```
package java.lang;  
  
public interface Comparable {  
    public int compareTo(Object o)  
}
```

(a) Prior to JDK 1.5

```
Comparable c = new Date();  
System.out.println(c.compareTo("red"));
```

(a) Prior to JDK 1.5



Runtime error



Generic Type

```
package java.lang;

public interface Comparable<T> {
    public int compareTo(T o)
}
```

(b) JDK 1.5

Improves reliability

```
Comparable<Date> c = new Date();
System.out.println(c.compareTo("red"));
```

(b) JDK 1.5



Compile error

